

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEMS

CLASS SCHEDULE

GROUP1 ICE-1&2	TUE 11.00 – 12.30(A202)	THU 11.00 – 12.30(A202)
GROUP2 CSE-1	MON 09.30 – 11.00(A202)	THU 09.30 – 11.00(A202)
GROUP3 CSE-2	MON 11.00 – 12.30(A202)	WED 11.00 – 12.30(A202)
GROUP4 CSE-3	TUE 09.30 – 11.00(A202)	WED 09.30 – 11.00(A202)

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

- 1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES**
- 2. STUDENTS COMING LATE TO THE CLASSES AFTER 9.30AM & 11AM WILL NOT BE ALLOWED TO ENTER THE CLASS**
- 3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS**
- 4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK**
- 5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY**
- 6. SLEEPING IN CLASS IS TOTALLY PROHIBITED**
- 7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS**
- 8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)**

SOC 3010

OPERATING SYSTEM

WEEK 1 LECTURE 2

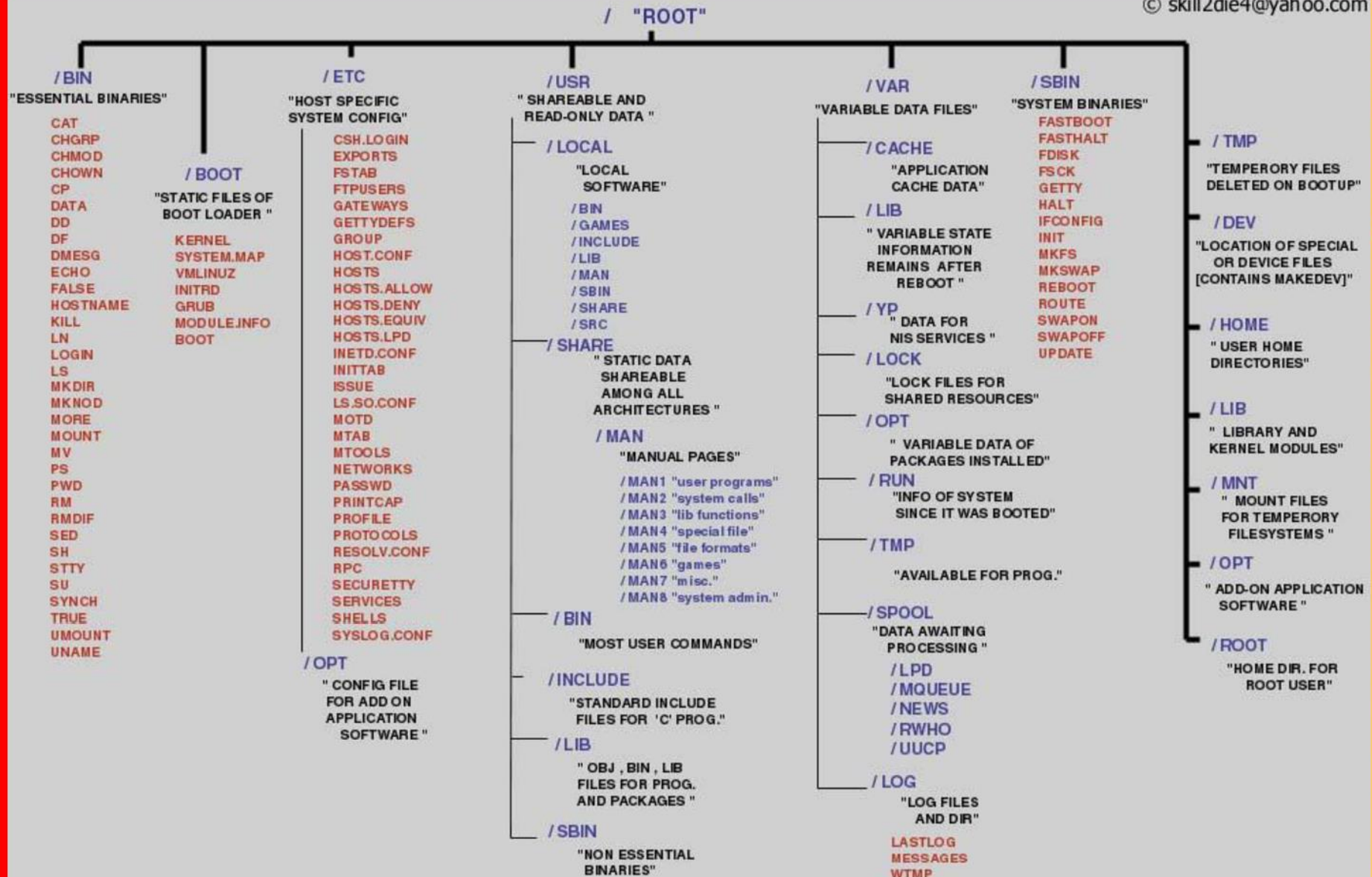
OVERVIEW OF UNIX/LINUX FILE SYSTEM

The Linux File System

- The Linux file system structure looks like an inverted tree structure upside down tree.
- You start with the root directory, denoted by /, at the top and work down through sub-directories underneath it.
- ★ Directories (a.k.a. “folders” in Windows) are collections of files and other directories.
- ★ Every directory has a parent except for the root directory.
- ★ Many directories have subdirectories.

The Linux File System

© skill2die4@yahoo.com



Overview of Unix File System

□ Files

- * A Unix file is an information container structured as a sequence of bytes; the kernel does not interpret the contents of a file.
- * From the user's point of view, files are organized in a tree-structured namespace.
- * All the nodes of the tree, except the leaves, denote directory names.
- * A directory node contains information about the files and directories just beneath it.
- * A file or directory name consists of a sequence of arbitrary ASCII characters, with the exception of / and of the null character \0.
- * **Most filesystems place a limit on the length of a filename, typically no more than 255 characters.**
- * The directory corresponding to the root of the tree is called the root directory.
- * By convention, its name is a slash (/).
- * Names must be different within the same directory, but the same name may be used in different directories.

Regular Files

- ✱ **A regular file contains arbitrary data**
- ✱ Applications often distinguish between *text files* and *binary files*
 - Text files are regular files with only ASCII or Unicode characters
 - Binary files are everything else
 - ✱ e.g., object files, JPEG images
 - Kernel doesn't know the difference!
- ✱ Text file is sequence of *text lines*
 - Text line is sequence of chars terminated by *newline char* ('`\n`')
 - ✱ Newline is 0xa, same as ASCII line feed character (LF)
- ✱ End of line (EOL) indicators in other systems
 - Linux and Mac OS: '`\n`' (0xa)
 - ✱ line feed (LF)
 - Windows and Internet protocols: '`\r\n`' (0xd 0xa)
 - ✱ Carriage return (CR) followed by line feed (LF)



Directories

- ★ Directory consists of an array of *links*
 - Each link maps a *filename* to a file
- ★ Each directory contains at least two entries
 - . (dot) is a link to itself
 - .. (dot dot) is a link to *the parent directory* in the *directory hierarchy*
- ★ Commands for manipulating directories
 - **mkdir**: create empty directory
 - **ls**: view directory contents
 - **rmdir**: delete empty directory

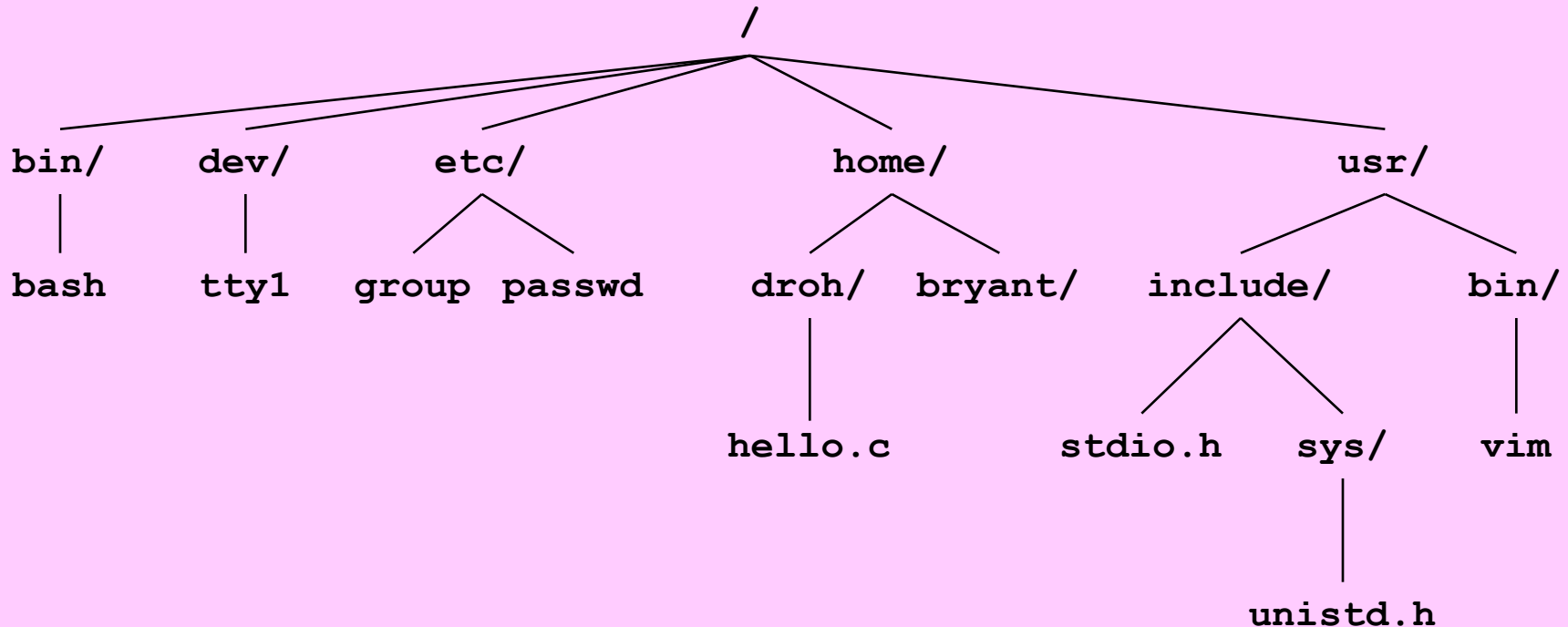
Overview of Unix File System

□ Files

- ★ Unix associates a current working directory with each process; it belongs to the process execution context, and it identifies the directory currently used by the process.
- ★ To identify a specific file, the process uses a pathname, which consists of slashes alternating with a sequence of directory names that lead to the file.
- ★ If the first item in the pathname is a slash, the pathname is said to be absolute, because its starting point is the root directory.
- ★ Otherwise, if the first item is a directory name or filename, the pathname is said to be relative, because its starting point is the process's current directory.
- ★ While specifying filenames, the notations "." to denote the current working directory and ".." to denote parent directory are also used.
- ★ If the current working directory is the root directory, "." and ".." coincide.

Directory Hierarchy

- ★ All files are organized as a hierarchy anchored by root directory named / (slash)



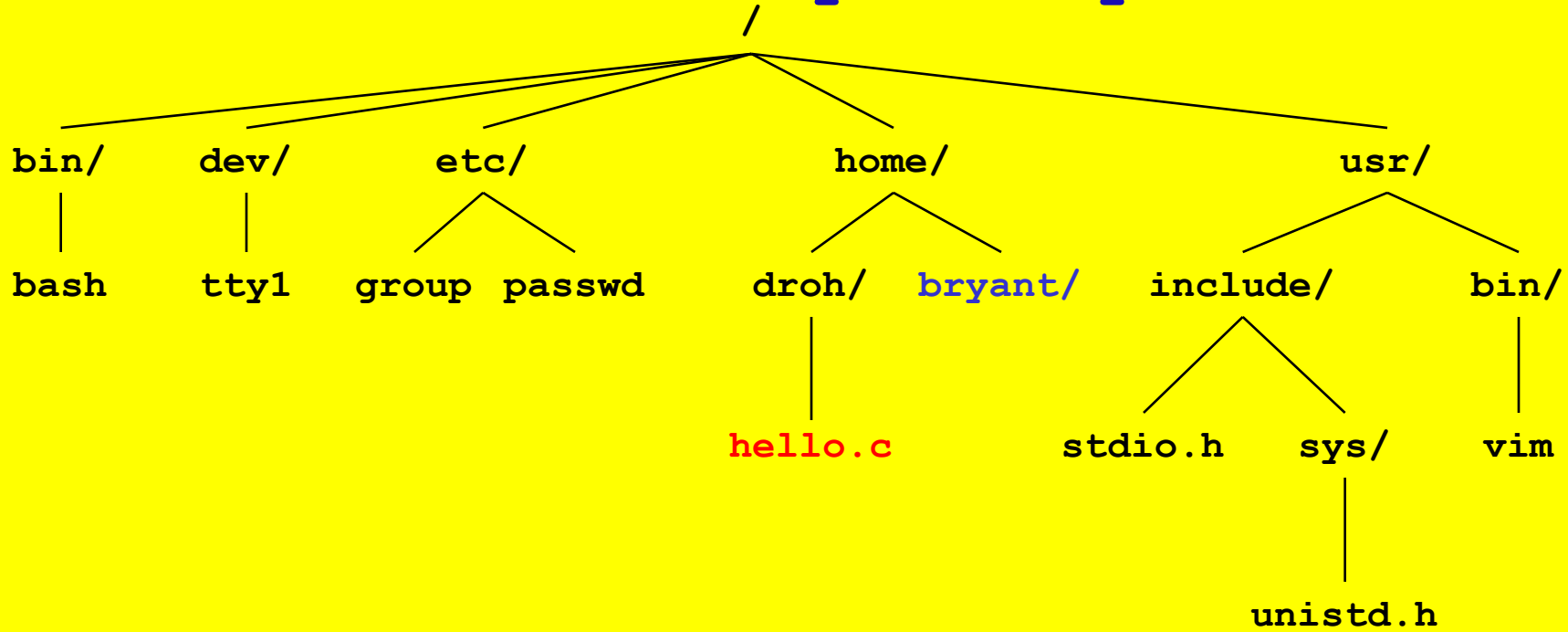
- ★ Kernel maintains *current working directory (cwd)* for each process
 - Modified using the `cd` command

Pathnames



- * Locations of files in the hierarchy denoted by *pathnames*
 - *Absolute pathname* starts with '/' and denotes path from root Example : `/home/droh/hello.c`
 - *Relative pathname* denotes path from current working directory

Example: `../droh/hello.c` if the current directory is bryant



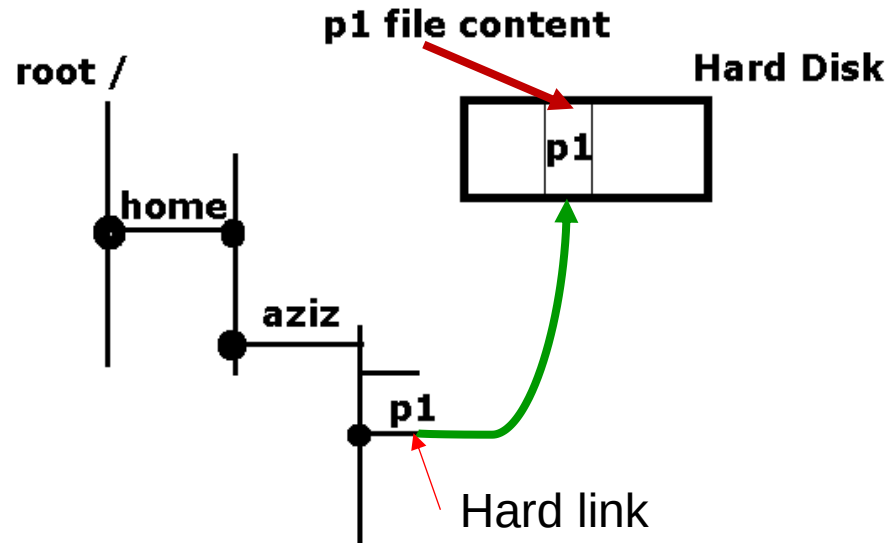
HARD & SOFT LINKS

□ HARD LINKS

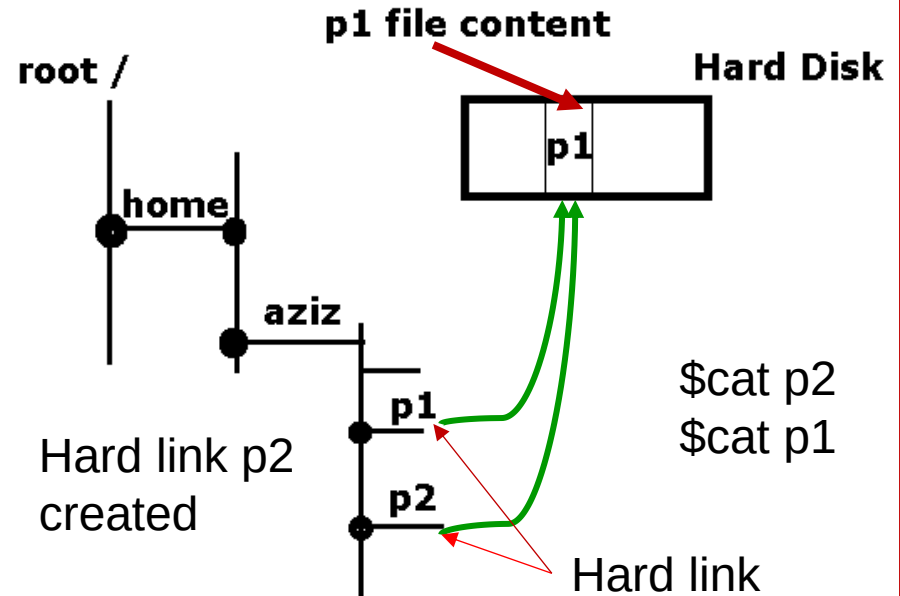
- * A filename included in a directory is called a file hard link, or more simply, a link.
- * The same file may have several links included in the same directory or in different ones, so it may have several filenames.
- * **The Unix command:**
\$ ln p1 p2
is used to create a new hard link that has the pathname p2 for a file identified by the pathname p1.
- * **Hard links have two limitations:**
 - It is not possible to create hard links for directories.
 - Links can be created only among files included in the same filesystem.
- * To overcome these limitations, **soft links (also called symbolic links) are used.**

HARD LINK - EXAMPLE

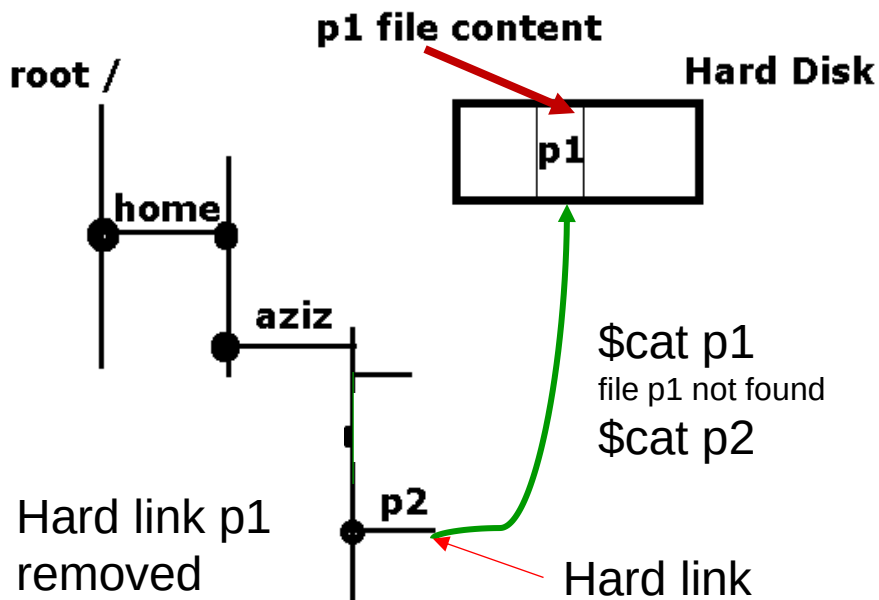
a) file p1 created in directory aziz



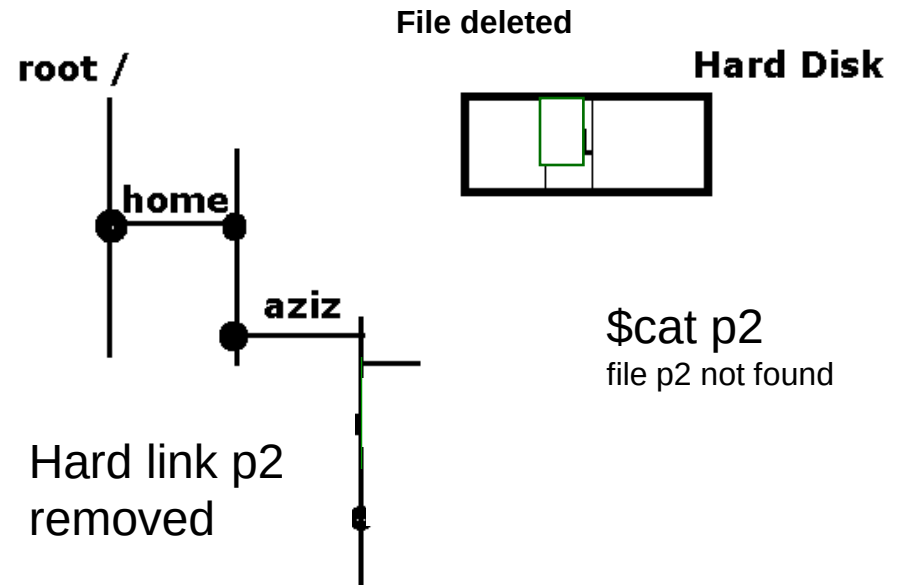
b) `$ln p1 p2`



c) `$rm p1`



d) `$rm p2`



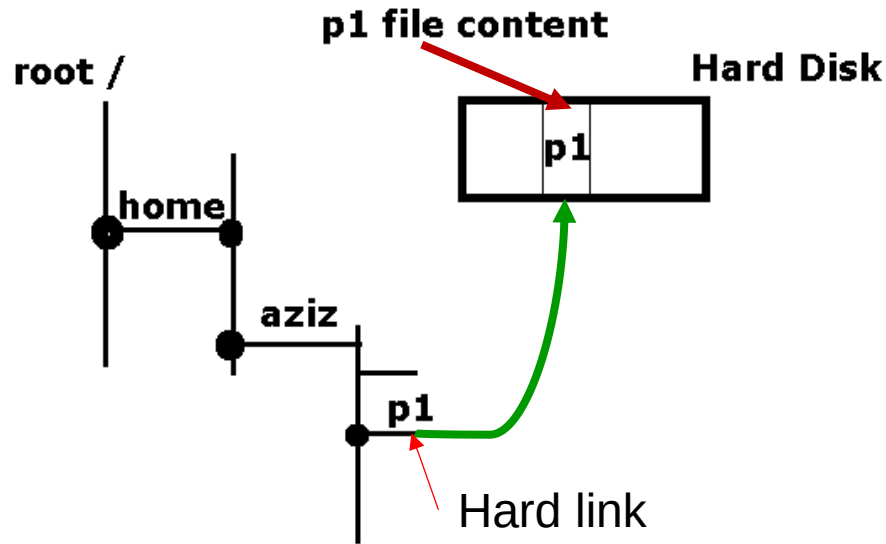
HARD & SOFT LINKS

□ SYMBOLIC LINKS (SOFT LINKS)

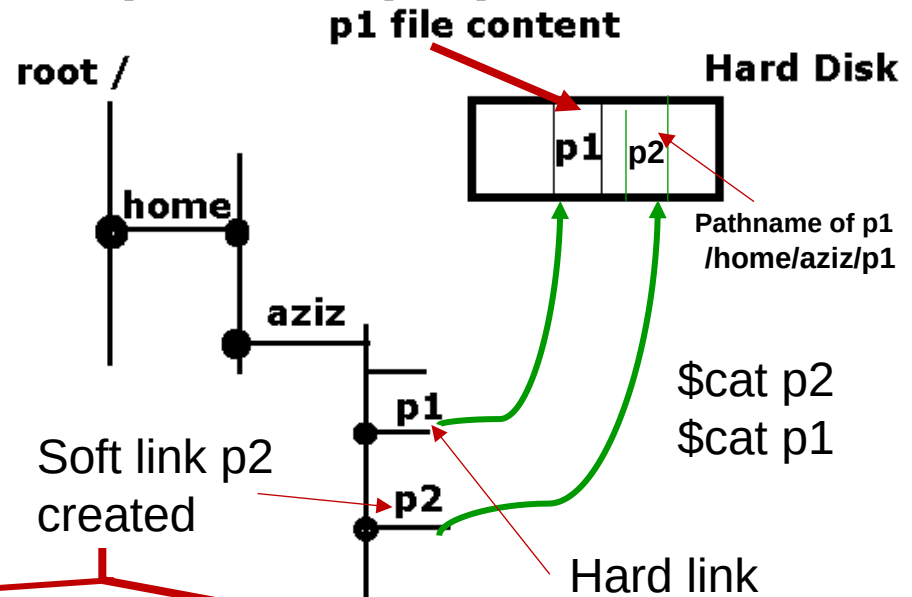
- * Symbolic links are short files that contain an arbitrary pathname of another file.
- * The pathname may refer to any file or directory located in any filesystem; it may even refer to a non-existent file.
- * **The Unix command:**
\$ ln -s p1 p2
creates a new soft link with pathname p2 that refers to pathname p1.
- * When this command is executed, the filesystem extracts the directory part of p2 and creates a new entry in that directory of type symbolic link, with the name indicated by p2.
- * This new file contains the name indicated by pathname p1.
- * This way, each reference to p2 can be translated automatically into a reference to p1.

SOFT LINK - EXAMPLE

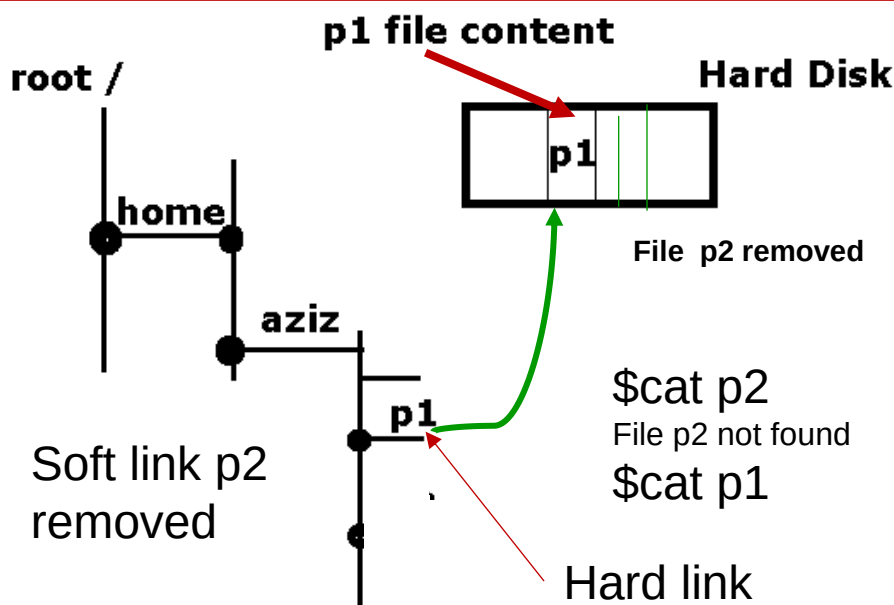
a) file p1 created in directory aziz



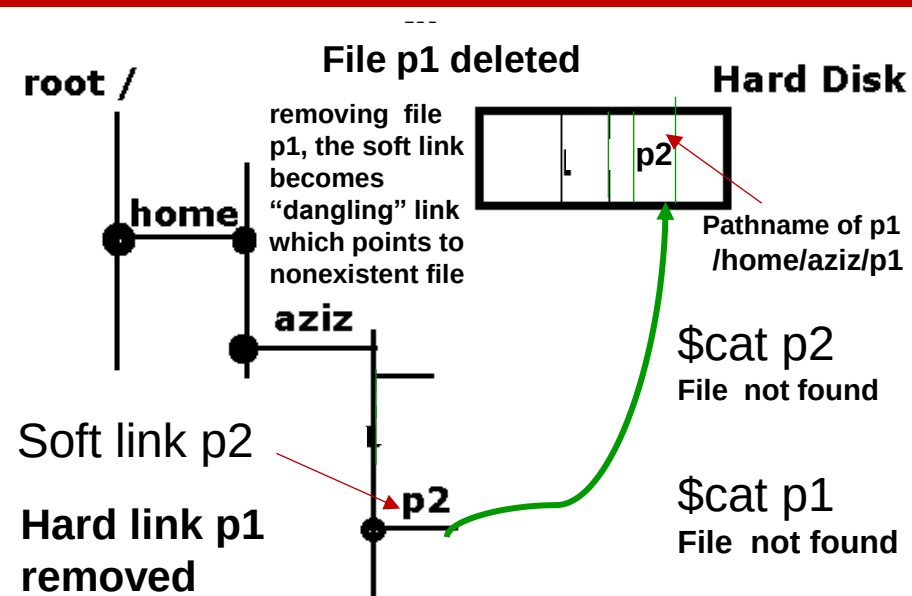
b) `$ln -s p1 p2`



c) `$rm p2`



c) `$rm p1`

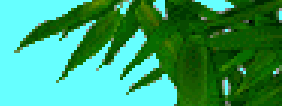


FILE TYPES

□ File Types

- * **Unix files may have one of the following types:**
 - **Regular file**
 - **Directory**
 - **Symbolic link**
 - **Block-oriented device file**
 - **Character-oriented device file**
 - **Pipe and named pipe (also called FIFO)**
 - **Socket**
- * **The first three file types are constituents of any Unix filesystem.**
- * **Device files are related both to I/O devices, and to device drivers integrated into the kernel.**
- * **Pipes and sockets are special files used for interprocess communication**

File Descriptor - Inode



□ File Descriptor - Inode

- * Unix makes a clear distinction between the contents of a file and the information about a file.
- * With the exception of device files and files of special filesystems, each file consists of a sequence of bytes.
- * The file does not include any control information, such as its length or an end-of-file (EOF) delimiter.
- * All information needed by the filesystem to handle a file is included in a data structure called an inode.
- * Each file has its own inode, which the filesystem uses to identify the file and contains the following attributes:
 - File type
 - Number of hard links associated with the file•
 - File length in bytes
 - Device ID (i.e., an identifier of the device containing the file)
 - Inode number that identifies the file within the filesystem
 - UID of the file owner
 - User group ID of the file
 - Several timestamps that specify the inode status change time, the last access time, and the last modify time
 - Access rights and file mode

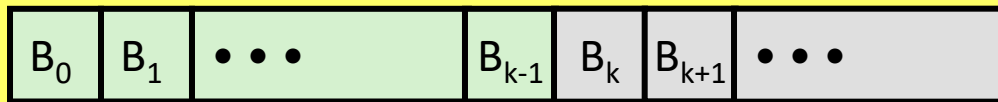


Unix/Linux I/O Overview

- ★ A Linux *file* is a sequence of m bytes:
 - $B_0, B_1, \dots, B_k, \dots, B_{m-1}$
- ★ Cool fact: All I/O devices are represented as files:
 - `/dev/sda2` (`/usr` disk partition)
 - `/dev/tty2` (terminal)
- ★ Even the kernel is represented as a file:
 - `/boot/vmlinuz-3.13.0-55-generic` (kernel image)
 - `/proc` (kernel data structures)
- All I/O devices, such as networks, disks, and terminals, are modeled as files, and all input and output is performed by reading and writing the appropriate files

Unix I/O Overview

- * Elegant mapping of files to devices allows kernel to export simple interface called *Unix I/O*:
 - Opening and closing files
 - * `open()` and `close()`
 - Reading and writing a file
 - * `read()` and `write()`
 - Changing the *current file position* (seek)
 - * indicates next offset into file to read or write
 - * `lseek()`



Current file position = k

Opening Files

- ❑ **Opening files: `open()`**
- An application announces its intention to access an I/O device by asking the kernel to open the corresponding file using `open()` system call.
- The kernel returns a small nonnegative integer, called a descriptor, that identifies the file in all subsequent operations on the file.
- ★ Each process created by a Linux shell begins life with three open files associated with a terminal:
 - 0: standard input (`stdin`)
 - 1: standard output (`stdout`)
 - 2: standard error (`stderr`)
- The header file `<unistd.h>` defines constants `STDIN_FILENO`, `STDOUT_FILENO`, and `STDERR_FILENO`, which can be used instead of the explicit descriptor values.

Opening Files



- ★ A process opens an existing file or creates a new file by calling the open function:

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int open(char *filename, int flags, mode_t mode);
```

Returns: new file descriptor if OK, -1 on error

- The open function converts a filename to a file descriptor and returns the descriptor number.
- The descriptor returned is always the smallest descriptor that is not currently open in the process.
- The flags argument indicates how the process intends to access the file:
 - ❖ **O_RDONLY: Reading only**
 - ❖ **O_WRONLY: Writing only**
 - ❖ **O_RDWR: Reading and writing**

Opening Files

For example, here is how to open an existing file for reading:

```
fd = Open("foo.txt", O_RDONLY, 0);
```

- The flags argument can also be or'd with one or more bitmasks that provide additional instructions for writing:

O_CREAT: If the file doesn't exist, then create a truncated (empty) version of it.

O_TRUNC: If the file already exists, then truncate it.

O_APPEND: Before each write operation, set the file position to the end of the file.

- For example, here is how you might open an existing file with the intent of appending some data:

```
fd = Open("foo.txt", O_WRONLY|O_APPEND, 0);
```

- The mode argument specifies the access permission bits of new files.

Opening Files

- The mode argument specifies the access permission bits of new files.
- The potential users of a file fall into three classes:
 - The user who is the owner of the file - u
 - The users who belong to the same group as the file, not including the owner - g
 - All remaining users (others) – o
- There are three types of access rights(access mode) -- read, write, and execute for each of these three classes. Thus, the set of access rights associated with a file consists of nine different binary flags

Opening Files

- The mode argument specifies the access permission bits of new files.
- The symbolic names for these bits are shown in Figure.

Mask	Description
S_IRUSR	User (owner) can read this file
S_IWUSR	User (owner) can write this file
S_IXUSR	User (owner) can execute this file
S_IRGRP	Members of the owner's group can read this file
S_IWGRP	Members of the owner's group can write this file
S_IXGRP	Members of the owner's group can execute this file
S_IROTH	Others (anyone) can read this file
S_IWOTH	Others (anyone) can write this file
S_IXOTH	Others (anyone) can execute this file

These Access permission bits are defined in `sys/stat.h`.

Opening Files

- As part of its context, each process has a **umask** that is set by calling the **umask function**.
- When a process creates a new file by calling the open function with some mode argument, then the access permission bits of the file are set to **mode & ~umask**.
- For example, suppose we are given the following default values for mode and umask:

```
#define DEF_MODE  S_IRUSR|S_IWUSR|S_IRGRP|S_IWGRP|S_IROTH|S_IWOTH  
#define DEF_UMASK  S_IWGRP|S_IWOTH
```
- Then the following code fragment creates a new file in which the owner of the file has read and write permissions, and all other users have read permissions:

```
umask(DEF_UMASK);  
fd = Open("foo.txt", O_CREAT|O_TRUNC|O_WRONLY, DEF_MODE);
```

Reading and writing files

□ Reading and writing files

- A read operation copies $n > 0$ bytes from a file to memory, starting at the current file position k , and then incrementing k by n .
- Given a file with a size of m bytes, performing a read operation when $k \geq m$ triggers a condition known as end-of-file (EOF), which can be detected by the application.
- There is no explicit “EOF character” at the end of a file.
- Similarly, a write operation copies $n > 0$ bytes from memory to a file, starting at the current file position k , and then updating k .
- Reading and writing a file **read()** and **write()**

Reading and writing files

❑ Reading and writing files

- Applications perform input and output by calling the read and write functions, respectively.

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t n);
```

Returns: number of bytes read if OK, 0 on EOF, -1 on error

```
ssize_t write(int fd, const void *buf, size_t n);
```

Returns: number of bytes written if OK, -1 on error

- The read function copies at most n bytes from the current file position of descriptor fd to memory location buf.
- A return value of -1 indicates an error, and a return value of 0 indicates EOF.
- Otherwise, the return value indicates the number of bytes that were actually transferred.
- ★ The write function copies at most n bytes from memory location buf to the current file position of descriptor fd.

Reading Files

- ★ Reading a file copies bytes from the current file position to memory, and then updates file position

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open file fd ... */
/* Then read up to 512 bytes from file fd */
if ((nbytes = read(fd, buf, sizeof(buf))) < 0) {
    perror("read");
    exit(1);
}
```

- ★ Returns number of bytes read from file `fd` into `buf`
 - Return type `ssize_t` is signed integer
 - `nbytes < 0` indicates that an error occurred

Writing Files

- ✱ Writing a file copies bytes from memory to the current file position, and then updates current file position

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open the file fd ... */
/* Then write up to 512 bytes from buf to file fd
*/
if ((nbytes = write(fd, buf, sizeof(buf))) < 0) {
    perror("write");
    exit(1);
}
```

- ✱ Returns number of bytes written from `buf` to file `fd`
 - `nbytes < 0` indicates that an error occurred

Reading and writing files

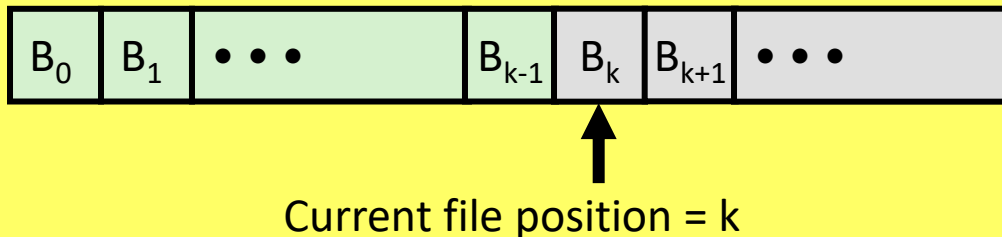
- * Figure shows a program that uses read and write calls to copy the standard input to the standard output, 1 byte at a time

```
int main(void)
{
    char c;
    while(read(STDIN_FILENO, &c, 1) != 0)
        write(STDOUT_FILENO, &c, 1);
    exit(0);
}
```

Changing the current file position

□ Changing the current file position

- The kernel maintains a file position k , initially 0, for each open file.
- The file position is a byte offset from the beginning of a file.
- An application can set the current file position k explicitly by performing a seek operation
 - Changing the **current file position** (seek)
 - * indicates next offset into file to read or write
 - * `lseek()`



Changing the current file position

- * Input and output are normally sequential: each read or write takes place at a position in the file right after the previous one.
- * When necessary, however, a file can be read or written in any arbitrary order.
- * The system call `lseek` provides a way to move around in a file without reading or writing any data:

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
long lseek(int fd, long offset, int origin);
```

Return value from `lseek` is a long that gives the new position in the file, or -1 if an error occurs.

- * sets the current position in the file whose descriptor is `fd` to **offset**, which is taken relative to the location specified by **origin**.
- * Subsequent reading or writing will begin at that position.
- * **origin** can be 0, 1, or 2 to specify that offset is to be measured from the beginning (`SEEK_SET`), from the current position (`SEEK_CUR`), or from the end of the file (`SEEK_END`) respectively

Changing the current file position

- ★ For example, to append to a file (the redirection >> in the UNIX shell, or "a" for fopen), seek to the end before writing:

```
lseek(fd, 0L, 2);
```

- ★ To get back to the beginning ("rewind"),

```
lseek(fd, 0L, 0);
```

- ★ Notice the 0L argument; it could also be written as (long) 0 or just as 0 if lseek is properly declared.
- ★ With lseek, it is possible to treat files more or less like arrays, at the price of slower access.

Closing Files

❑ Closing files

- When an application has finished accessing a file, it informs the kernel by asking it to close the file.
- The kernel responds by freeing the data structures it created when the file was opened and restoring the descriptor to a pool of available descriptors.
- When a process terminates for any reason, the kernel closes all open files and frees their memory resources.
- Finally, a process closes an open file by calling the close function.

```
#include <unistd.h>
```

```
int close(int fd);
```

Returns: zero if OK, -1 on error

- Closing a descriptor that is already closed is an error.

Renaming and deleting a file

❑ Renaming and deleting a file

- To rename or delete a file, a process does not need to open it. Indeed, such operations do not act on the contents of the affected file, but rather on the contents of one or more directories.
- For example, the system call:
`res = rename(oldpath, newpath);`
changes the name of a file link,
- While the system call:
`res = unlink(pathname);`
decreases the file link count and removes the corresponding directory entry. The file is deleted only when the link count assumes the value 0.

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

